

Resolución de problemas y algoritmos

Dra. Jessica Andrea Carballido

jac@cs.uns.edu.ar



Dpto. de Ciencias e Ingeniería de la Computación

UNIVERSIDAD NACIONAL DEL SUR

CLASE 15

Manejo de memoria

Program intercambios;

var a, b: integer;

procedure SWAP(var x, y: integer);

var aux: integer;

begin

aux:=x;

x:=y;

y:=aux;

end;

begin

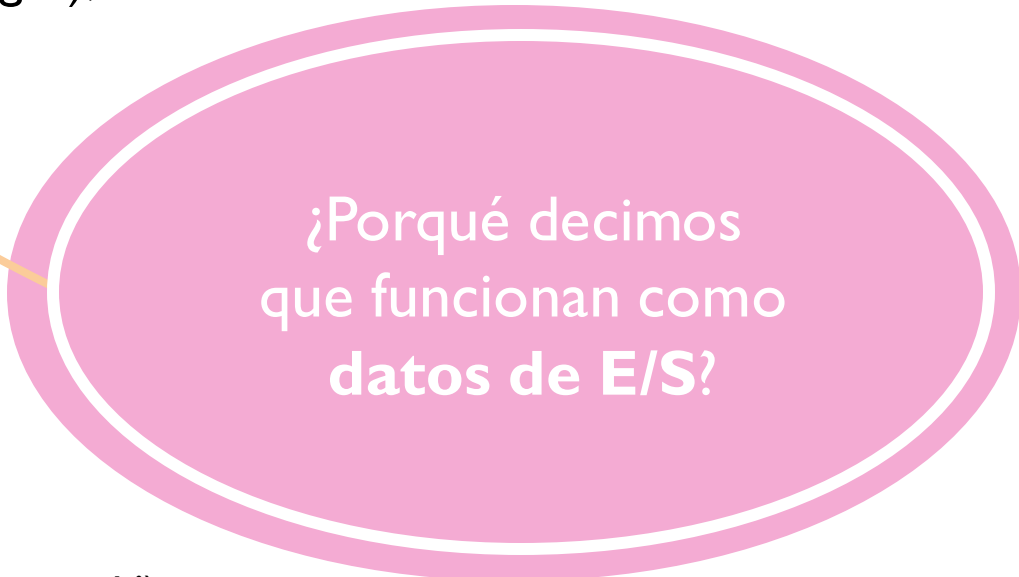
Writeln('Ingrese un valor para a y uno para b');

Readln(a, b);

SWAP(a, b);

Writeln('El valor de a es:', a, ' y el de b es:', b);

End.



¿Porqué decimos
que funcionan como
datos de E/S?



*Lo responden haciendo
una traza completa*



UN POCO
DE TEORÍA



Visibilidad, alcance y tiempo de vida



Identificadores: denotan constantes, tipos, variables, parámetros, subprogramas, etc.

El **alcance** (scope) de un identificador es la **zona** del programa donde el identificador es **visible**.

Un identificador es **visible** en un bloque ejecutable si **puede usarse** en ese bloque ejecutable.

Bloque/bloques ejecutables
(entre begin-end)



Visibilidad, alcance y tiempo de vida



Identificadores: denotan constantes, tipos, variables, parámetros, subprogramas, etc.

El **tiempo de vida** de un identificador es el período que transcurre desde que se reserva su locación en **memoria** hasta que la misma se elimina.

La entidad denotada por un identificador puede estar **viva** **y no ser visible** en una parte del programa.

Es decir, está viva si tiene asociado un lugar en la pila.



Visibilidad, alcance y tiempo de vida



La entidad denotada por una referencia a un identificador depende de la visibilidad y de las reglas de alcance del lenguaje.

Alcance estático

El alcance de los identificadores se define de acuerdo a cómo está escrito el programa.
Toda entidad es visible en el bloque que se declara y en sus bloques internos.



Ambientes de referenciamiento

```
program parampyf;  
var pa1, pa2: integer;  
function f(x: integer): integer;  
begin  
    ...  
end;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
    ...  
end;  
begin  
    ...  
end.
```

Identificadores
que se pueden
referenciar

El ambiente de referenciamiento
del programa principal
se denomina **entorno global**.



Ambientes de referenciamiento



- Los identificadores declarados en el programa principal se llaman **globales**. Están **vivos a lo largo de toda la ejecución** del programa (siempre permanecen almacenados en la pila), aunque no siempre son visibles.
- El bloque ejecutable del **programa principal** puede acceder a las variables **pa1** y **pa2** y también **puede invocar a la función f y al procedimiento p**.
- El procedimiento **p** **puede invocar a f y también puede invocarse a sí mismo**, como veremos más adelante. En su bloque ejecutable puede acceder a **pf1**, **pf2**, **local** y a las variables globales (**esto último no lo permitiremos**).
- La función **f** **sólo puede invocarse a sí misma**, el procedimiento **p** **no es visible para f**. En su bloque ejecutable puede acceder a **x** y a las variables globales (**esto último no lo permitiremos**).



Ambientes de referenciamiento

Aunque Pascal permite que el procedimiento p y la función f accedan a las variables globales pa1 y pa2, la **programación modular** recomienda no hacerlo y **adoptaremos esa recomendación**.

La programación modular RECOMIENDA que el **bloque ejecutable de cada función o procedimiento acceda únicamente a sus parámetros formales y variables locales**.



En RPA, en los procedimientos y funciones, el uso de variables globales o variables declaradas en un entorno no local **será considerado un error**.

Ambientes de referenciamiento



- La variable x es el parámetro formal de la función f y solo es visible en el bloque ejecutable de la función f .
- Las variables $pf1$, $pf2$ y $local$ solo son visibles en el bloque ejecutable del procedimiento p .
- Las variables $pa1$ y $pa2$ (globales) son visibles en todos los bloques ejecutables, aunque solo las usaremos en el bloque ejecutable del programa principal.
 - **IMPORTANTE!** Si declaramos una nueva variable $pa1$ local a la función f , la variable $pa1$ del programa principal deja de ser visible en el bloque ejecutable de f ($pa1$ global es un ejemplo de entidad que está viva y no es visible).



Ambientes de referenciamiento

```
program parampyf;  
var pa1, pa2: integer;  
function f(x: integer): integer;  
begin  
    ...  
end;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
    ...  
end;  
begin  
    ...  
end.
```

Entorno local de f:
parámetros formales
+ variables locales
+ sus subprogramas



Ambientes de referenciamiento

```
program parampyf;  
var pa1, pa2: integer;  
function f(x: integer): integer;  
begin  
    ...  
end;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
    ...  
end;  
begin  
    ...  
end.
```

Entorno local de p:
parámetros formales
+ variables locales
+ sus subprogramas

Además siempre se pueden utilizar todos los identificadores predefinidos.

Visibilidad, alcance y tiempo de vida



```
program parampyf;  
var pa1, pa2: integer;  
function f(x: integer): integer;  
begin  
    ... // identificadores visibles: x, f, pa1 y pa2.  
end;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
    ...  
end;  
begin  
    ...  
end.
```

VERY IMPORTANT

Si desde p invocamos a f, tanto pf1 como pf2 y local son identificadores de variables que **están vivas** pero no son visibles durante la ejecución de f.



Visibilidad, alcance y tiempo de vida



```
program parampyf;  
  var pa1, pa2: integer;  
  procedure p(pf1: integer; var pf2: integer);
```

```
    var local: integer;
```

```
    function f(x: integer): integer;
```

```
      var pf1: integer;
```

```
      begin
```

... // visibles: **x**, **pf1** (declarado en **f**), **f**, **local**, **pf2**, **p**, **pa1** y **pa2**.

```
    end;
```

```
  begin
```

... // visibles: **pf1**, **pf2**, **local**, **f**, **p**, **pa1** y **pa2**.

```
end;
```

```
begin
```

... // visibles: **p**, **pa1**, **pa2**

```
end.
```

El ambiente “no local” aparece cuando tenemos un sub-programa declarado dentro de otro sub-programa.

local a f

no local

global

local a p

global

Cuando se hace referencia a un identificador, se busca a qué declaración corresponde en el siguiente orden:

entorno local → no local → global → predefinido.

Visibilidad, alcance y tiempo de vida



Corolarios:

- ✓ En cada primitiva, solo vamos a acceder a las variables del entorno local (parámetros formales y variables locales).
- ✓ Cada primitiva podrá invocarse a sí misma, a las declaradas en su entorno local o podrá invocar a las primitivas declaradas en el entorno global que hayan sido definidas con anterioridad.

